future internet

MDPI

Article

Ad-BBR: Enhancing Round-Trip Time Fairness and Transmission Stability in TCP-BBR

Mingjun Wang [1,2], Xuezhi Zhang [1,2], Feng Jing [2,3] and Mei Gao [2,3,*]

[1] School of Automation and Information Engineering, Xi'an University of Technology, Xi'an 710048, China
[2] Shaanxi Key Laboratory of Intelligence Coordination Networks, Xi'an 710106, China
[3] Test and Training Base, National University of Defense Technology, Xi'an 710106, China
* Correspondence: gao_mei@nudt.edu.cn

Abstract: The rapid development of wireless network technology and the continuous evolution of network service demands have raised higher requirements for congestion control algorithms. In 2016, Google proposed the Bottleneck Bandwidth and Round-trip propagation time (BBR) congestion control algorithm based on the Transmission Control Protocol (TCP) protocol. While BBR offers lower latency and higher throughput compared to traditional congestion control algorithms, it still faces challenges. These include the periodic triggering of the ProbeRTT phase, which impairs data transmission efficiency, data over-injection caused by the congestion window (CWND) value-setting policy, and the difficulty of coordinating resource allocation across multiple concurrent flows. These limitations make BBR less effective in multi-stream competition scenarios in high-speed wireless networks. This paper analyzes the design limitations of the BBR algorithm from a theoretical perspective and proposes the Adaptive-BBR (Ad-BBR) algorithm. The Ad-BBR algorithm incorporates real-time RTT and link queue-state information, introduces a new RTprop determination mechanism, and implements a finer-grained, RTT-based adaptive transmission rate adjustment mechanism to reduce data over-injection and improve RTT fairness. Additionally, the ProbeRTT phase-triggering mechanism is updated to ensure more stable and smoother data transmission. In the NS3, 5G, and Wi-Fi simulation experiments, Ad-BBR outperformed all comparison algorithms by effectively mitigating data over-injection and minimizing unnecessary entries into the ProbeRTT phase. Compared to the BBRv1 algorithm, Ad-BBR achieved a 17% increase in throughput and a 30% improvement in RTT fairness, along with a 13% reduction in the retransmission rate and an approximate 20% decrease in latency.

Keywords: TCP congestion control; BBR; multi-flow competition; RTT fairness

check for updates

Academic Editor: Gianluigi Ferrari

Received: 13 March 2025
Revised: 16 April 2025
Accepted: 16 April 2025
Published: 22 April 2025

Citation: Wang, M.; Zhang, X.; Jing, F.; Gao, M. Ad-BBR: Enhancing Round-Trip Time Fairness and Transmission Stability in TCP-BBR. *Future Internet* **2025**, *17*, 189. https://doi.org/10.3390/fi17050189

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

With the rapid growth of Internet applications and the continuous evolution of wireless network technologies, such as IoT, 5G, and Wi-Fi, global data traffic is experiencing an explosive increase. Simultaneously, the network environment is becoming increasingly complex and dynamic, with factors like diverse user behaviors, constant updates to terminal devices, and frequent link condition fluctuations presenting unprecedented challenges to transport layer protocols and congestion control algorithms. Traditional congestion control algorithms based on packet loss (e.g., Reno and Cubic) often struggle to balance high throughput and low latency in high-bandwidth, low-latency, and highly dynamic scenarios while also exhibiting fairness issues in multi-stream competitive environments [1,2].

To address these bottlenecks, Google introduced the BBR algorithm in 2016 [3], which models link transmission by actively probing the bottleneck bandwidth (BtlBw) and the minimum round-trip propagation time (RTprop). BBR defines the maximum effective transmission rate of a link as the product of the BtlBw and RTprop (i.e., the bandwidth-delay product, BDP). By estimating these parameters in real time, BBR dynamically adjusts the sending rate to approximate the BDP, maximizing throughput while minimizing transmission latency. The BBR algorithm has been deployed across various Google platforms, such as B4 WAN and YouTube, and integrated into the Linux kernel.

In real high-speed wireless network environments (e.g., 5G and Wi-Fi), applications often involve both bandwidth-demanding long RTT streams and delay-sensitive short RTT streams, resulting in increasingly noticeable bandwidth competition between the two types of streams [4,5]. Due to their distinct characteristics and diverse causes, these streams have different QoS (Quality of Service) requirements and round-trip times (RTTs), posing significant challenges for the BBR algorithm. Specifically, to account for potential Acknowledge character (ACK) delays and queue buildup in real networks, BBR's control logic adjusts the congestion window (CWND) to be slightly larger than the calculated ideal value, preventing transmission stalls [6]. While this strategy helps avoid link idling, it also allows excess data to be injected into the network, leading to the phenomenon of data over-injection [7]. This not only causes queue buildup and increases the RTT but also heightens the risk of packet loss, ultimately affecting overall transmission performance and network stability. In terms of fairness, when multiple BBR flows share the same network bottleneck link, flows with different round-trip times (RTTs) are unable to allocate bandwidth fairly, as long RTT flows tend to obtain excessive bandwidth, while short RTT flows struggle to achieve reasonable transmission rates. Bandwidth allocation is, thus, heavily influenced by the RTTs of competing flows [7,8]. This phenomenon further limits the applicability of the BBR algorithm in multi-flow competitive environments [6]. Additionally, the BBR algorithm periodically enters the ProbeRTT phase without assessing its necessity, aiming to determine the minimum RTT with emptied queues. This results in bandwidth wastage and transmission interruptions, leading to unstable data transmission and undermining the original goals of the BBR algorithm [9].

To address these issues, the focus of current research is on effectively preventing data over-injection and reducing queue delay while preserving the high-throughput and low-latency advantages of BBR. Additionally, improving fairness in multi-stream competition to achieve smoother transmission has been a key challenge. However, most studies have concentrated on enhancing RTT fairness while largely overlooking throughput degradation and failing to thoroughly investigate the redundant triggering behavior of the ProbeRTT phase. Although Google's official versions—BBRv2 and BBRv3—aim to improve the adaptivity and fairness of the BBR algorithm, they suffer from significant throughput degradation compared to BBRv1 [10,11]. While these modifications do enhance congestion control, they also deteriorate overall link performance. Similarly, several unofficial variants, such as BBQ—which suppresses the sending rate of long RTT streams [12]—have been shown, through analysis and experiments [13,14], to possess inherent limitations. Moreover, BBR-PG [15] improves overall fairness but lacks fine-grained adjustment, resulting in persistent bandwidth wastage. To the best of our knowledge, few studies have successfully balanced RTT fairness with throughput performance, achieving significant fairness enhancements without substantially compromising throughput. This paper analyzes the limitations of BBR's design mechanisms from a theoretical perspective and proposes the Ad-BBR algorithm. This algorithm incorporates real-time RTT and link queue-state monitoring, as well as introduces a new RTprop determination mechanism and finer-grained adaptive rate adjustment based on the RTT to mitigate over-injection and improve RTT fair-

ness. Moreover, the ProbeRTT phase trigger mechanism is updated to enhance the stability and smoothness of data transmission. The proposed approach aims to offer a practical optimization solution for congestion control in high-speed wireless network environments. Inspired by these motivations, this paper presents the following contributions:

1. This paper analyzes the data over-injection phenomenon, the long- and short-flow RTT unfairness, and some of the unnecessary ProbeRTT phase triggers in the BBR congestion control algorithm through formula derivation and theoretical proofs.
2. The Ad-BBR algorithm is proposed to mitigate the data over-injection issue and enhance RTT fairness in multi-stream contention scenarios. This is achieved by replacing the RTprop with a dynamically varying *nRTT* and adaptively adjusting the sending rate based on the real-time RTT and *nRTT*. Additionally, a new ProbeRTT phase entry logic is introduced to reduce unnecessary send-rate degradation and bandwidth waste.
3. The function and performance of the Ad-BBR algorithm are verified through NS3 and 5G and Wi-Fi simulation network experiments, and the proposed algorithm in this paper is evaluated in terms of the throughput, RTT fairness, retransmission rate, and latency, respectively, and the results show that Ad-BBR achieves higher throughput, better fairness, more stable data transmission, and improves the multi-stream contention in most cases of congestion control performance in multi-stream competition scenarios.

The remaining sections of this paper are organized as follows: Section 2 reviews existing research on optimizing the BBR congestion control algorithm. Section 3 discusses the principles of the BBR algorithm and the motivation for its improvement. Section 4 provides a detailed description of the design and structure of Ad-BBR. Section 5 presents experimental results and analysis, while Section 6 concludes the paper.

2. Related Work

In 2016, Google introduced the BBR congestion control algorithm, which redefined the concept of congestion control [3]. It introduced the delayed bandwidth product (BDP) and calculated the BDP value by alternately measuring the link's BtlBw and RTprop, ensuring the amount of inflight data remains close to the BDP value. This approach also solved the problem of identifying Kleinrock's optimal operating point [16]. BBR's operational method fundamentally differs from traditional algorithms that rely on packet loss, marking a shift in congestion control strategies. As a policy-based algorithm, BBR uses a feedback-driven mechanism that dynamically adjusts the congestion window in real time, optimizing the use of available bandwidth to improve performance.

However, many studies have shown that BBR seems to have some unfairness problems. Kim et al. [17] tested its fairness by building a testbed and observed that long RTT streams consistently outperform short RTT streams, especially during bandwidth allocation. To address this, they proposed a bottleneck queue buildup suppression algorithm that limits the flight cap to below 2 BDPs when a bottleneck queue forms, thereby suppressing excessive data transfers and minimizing the transmission queue. If no bottleneck is detected, the flight cap is set to 1 BDP. This was the first attempt to address the RTT fairness problem in the BBR algorithm. Although it improves RTT fairness to some extent, the convergence remains slow and unstable, and setting the upper limit to 1 BDP significantly impacts the total throughput. Ma et al. [12] found, through extensive experiments, that the probing duration of long RTT streams is always longer than that of short RTT streams. This results in long RTT streams dominating the transmission queue and increasing the likelihood of data pileup. To address this, they proposed an algorithm that limits the probing duration of long RTT streams, reducing their bandwidth pre-emption and consumption while improving

the competition fairness between long and short RTT streams. However, this algorithm is only effective when a small number of streams compete for bandwidth. Its performance significantly degrades when multiple streams compete simultaneously. The BBQ algorithm improves RTT fairness and reduces latency by reconfiguring the *pacing_gain* cycle's first phase with additional conditional constraints [12]. Meanwhile, Mahmud et al. [18] identified high packet loss, high retransmission rates, and throughput unfairness in the BBR algorithm. To address these issues, they developed an algorithm that reduces the congestion window (CWND) by half and decreases the transmission rate by 25% under high network congestion and packet loss. However, this approach performs poorly for delay-sensitive applications and services. Kim et al. observed that BBR algorithms often transmit excessive data, causing an imbalance in throughput across different RTT streams. To address this, they proposed the delay-aware BBR algorithm (DA-BBR), which improves RTT fairness and throughput by controlling the client's transmission rate and congestion window (CWND) based on the relationship between the RTT and RTprop [19]. Pan et al. noted that many existing algorithms use a fixed congestion window gain of 2, which results in smaller congestion windows for short RTT streams compared to long RTT ones. To overcome this, they developed the BBR adaptive congestion window (BBR-ace) algorithm, which dynamically adjusts the congestion window gain based on the sending rate and RTT [20]. Njogu et al. [21] proposed the BBR-EFRA algorithm to address competitive fairness issues between long and short RTT streams, as well as the potential for the malicious exploitation of fairness loopholes in buffer queue states. Yinker [22] introduced an algorithm for dynamically adjusting pacer_gain based on packet loss rates and congestion levels in wireless networks. Xu et al. [15] tackled RTT fairness issues in BBR under high-speed wireless network conditions, such as 5G and Wi-Fi. They developed a fluid model to analyze fairness differences and proposed real-time adjustments to ProbeBw and *pacing_gain* to improve fairness, maintain high throughput, and minimize retransmissions. However, their approach lacks fine-grained analysis of the regulation process, leading to unnecessary sending rate reductions and bandwidth waste in situations where rate adjustments are not needed. This issue arises because the adaptive regulation mechanism does not account for varying queueing states, preventing the link from operating at maximum efficiency. Deng Z et al. [23] proposed a BBR-based congestion control algorithm (S2B2C) along with a flow-path bounded priority scheduling mechanism (S2PS) in 2024. Their approach enhances MPQUIC throughput and fairness in heterogeneous networks through shared bottleneck detection and dynamic rate adjustment and mitigates network instability by employing mixed-integer programming for scheduling. However, the study did not verify BBRv2 compatibility and was limited to static topologies, which restricts the analysis of dynamic network adaptability. Tong V et al. [24] introduced a reinforcement learning-based adaptive congestion control algorithm, ABBR, designed to optimize network throughput, delay, and fairness by dynamically selecting pacing gain sequences among various BBR variants. However, ABBR exhibits poor fairness when deployed alongside loss-based algorithms (e.g., Cubic) and does not adequately address scheduling optimization via mixed-integer programming. In 2025, Han Z et al. [25] proposed the BBR-R algorithm, which enhances RTT fairness and reduces queuing delays in multi-stream scenarios by dynamically adjusting delay detection intervals and optimizing packet counts during the probing phase. However, the experiments were conducted in idealized scenarios—excluding large-scale multi-stream and complex network topologies—and the parameter settings were based on empirical observations rather than adaptive mechanisms. Moreover, the algorithm's fairness when coexisting with other congestion control methods was not sufficiently validated. Kihungi et al. [26] proposed the BBR-EBE+ algorithm, which enhances BBR's bandwidth estimation by accurately detecting and filtering excessively

high ACK rates and delaying bandwidth update estimations. This approach effectively reduces average queuing delays and modestly increases throughput, yet it still faces notable challenges in terms of RTT fairness.

Google has introduced the BBRv2 algorithm [27] to address the shortcomings of BBRv1. BBRv2 improves bandwidth estimation and adapts more accurately to the available link bandwidth while minimizing queue buildup. By incorporating packet loss rate and ECN (Explicit Congestion Notification) information, BBRv2 allows for quicker determination of when the StartUp phase ends, avoiding excessive queuing caused by over-probing. While BBRv2 offers better adaptability than BBRv1 [28,29], it still faces issues with fairness, especially in packet loss scenarios, which leads to competitive unfairness [10]. In response, Google introduced BBRv3 in July 2023 [11] to address fairness convergence and post-congestion recovery speed. BBRv3 provides better fairness than its predecessors when operating alongside traditional packet loss-based congestion control algorithms. However, its performance in high-speed wireless networks with varying RTTs remains unsatisfactory. Despite ongoing updates from Google, experiments have revealed that the long–short-flow RTT fairness problem persists in BBR congestion control applications [30]. Therefore, continued research is necessary to enhance RTT fairness and ensure stable, smooth data transmission based on real-time link states, ultimately contributing to the final version of BBR

3. Overview of BBR

3.1. BBR Algorithm

The operating mechanism of the BBR algorithm is shown in Figure 1:

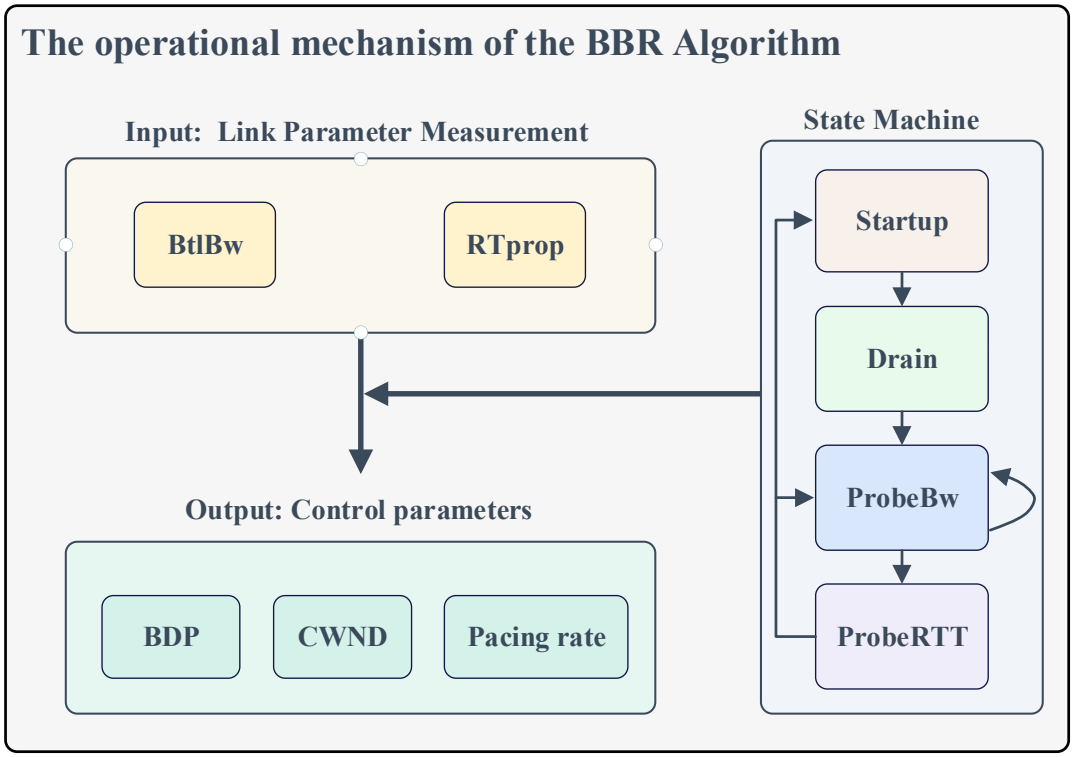


Figure 1. The operational mechanism of the BBR algorithm.

The BBR algorithm proposed by Google differs from traditional packet loss-based or delay-based probing models. It introduces the concept of a BDP, proposed by Westwood [31], which models the network link controlled by BBR as a physical pipe. In this analogy, the bottleneck bandwidth (BtlBw) represents the cross-sectional area of the pipe,

and the length of the link (RTprop) represents the length of the pipe. The ideal *BDP* is the product of these two values, i.e., $BDP = BtlBw^{*}RTprop$. The BDP represents the theoretical amount of "inflight" data (packets sent but not yet ACKed) that the link can hold, specifically the total number of unacknowledged packets. This includes the amount of inflight data. When the amount of inflight data is exactly equal to the BDP, the link is fully utilized, i.e., the following equation:

$$Inflight_data = BDP_{target} = BtlBw^{*}RTprop \quad (1)$$

At this point, the link reaches Kleinrock's optimal operating point [32], as shown in Figure 2.

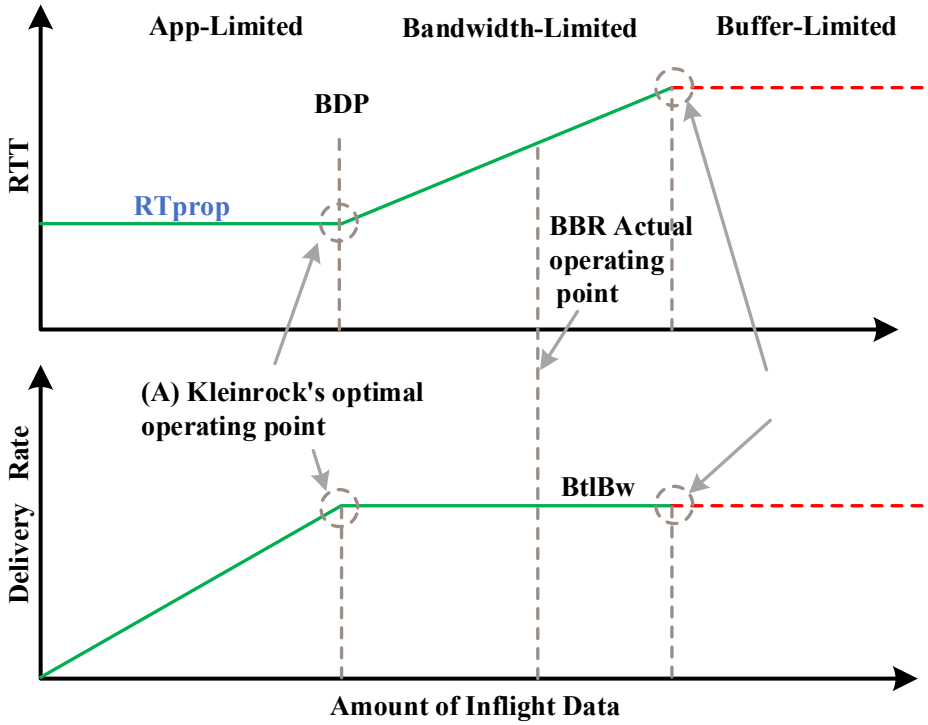


Figure 2. The optimal operating point of BBR and the loss-based operating point of congestion control algorithms.

To achieve Kleinrock's optimal operating point, accurate measurement and estimation of the BtlBw and RTprop are essential. Since both values can vary throughout the data transmission process, BBR must continuously estimate these dynamic changes.

The RTT value (the time interval between sending a packet and receiving its acknowledgment) at any given moment in a BBR-controlled TCP link may consist of the link's intrinsic delay RTprop and the queuing delay η_t, as shown in Equation (2):

$$RTT_t = RTprop + \eta_t \quad (2)$$

The *RTprop*, being a physical property of the link, only changes when the path changes. Therefore, the minimum RTT at time t, which is involved in subsequent operations, can be replaced by the unbiased estimator of *RTprop* at time t, as shown in Equation (3):

$$\min(RTT_t) = RTprop_t = RTprop + \min(\eta_t)\ \forall t \in [t - W_R, t] \quad (3)$$

where W_R represents the running time of the time window (ranging from several seconds to a few minutes).

On the other hand, the measurement of the BtlBw is determined by evaluating the data transmission rate, as the average transmission rate between the transmitter and the ACK satisfies Equation (4):

$$deliveryRate = \frac{\Delta delivered}{\Delta t} \tag{4}$$

The bottleneck rate, calculated as the ratio of the amount of data transmitted to the time taken, also represents the upper limit of the BtlBw, allowing the maximum transmission rate within this window to serve as an unbiased estimate of the BtlBw, as shown in Equation (5):

$$BtlBw = max(deliveryRate_t) \; \forall t \in [t - W_B, t] \tag{5}$$

BBR considers the following:

- The minimum RTT measured over the past ten seconds as the RTprop;
- The maximum transmission rate observed within the most recent ten RTT intervals as BtlBw.

Based on these parameters, BBR calculates the following:

- Bandwidth-delay product (BDP);
- Pacing rate;
- Congestion window (CWND).

Notably, the calculations of the pacing rate and CWND incorporate the scaling factors *pacing_gain* and *cwnd_gain*, respectively. Their computational formulas are detailed in Equations (6) and (7):

$$Pacing\ Rate = pacing_gain^* BtlBw \tag{6}$$

$$CWND = cwnd_gain^* BDP = cwnd_gain^* BtlBw^* RTprop \tag{7}$$

The BBR algorithm's state machine module consists of four phases: StartUp, Drain, ProbeBw, and ProbeRTT.

StartUp phase: The sender employs a mechanism analogous to the slow-start algorithm, where both *pacing_gain* and *cwnd_gain* (set to $2/\ln 2 \approx 2.89$) act as multiplicative factors to exponentially increase the sending rate for probing the maximum bottleneck bandwidth of the network. However, unlike traditional slow-start, BBR's StartUp phase incorporates three consecutive rounds of multiplicative rate increases and transitions to the Drain phase only when bandwidth growth is deemed ineffective, ensuring no premature exit from the probing phase. By this point, the system's operating state has already shifted to the right side of Kleinrock's optimal operating point (indicating persistent queue buildup).

Drain phase: In this phase, the sender dynamically adjusts the transmission rate by applying the multiplicative inverse of the startup parameter (set to $\ln 2/2$) to systematically drain potential queues created during the StartUp phase. This rate-limiting mechanism persists until the inflight data volume falls below or equals the estimated bandwidth-delay product (1*BDP), at which point the algorithm transitions into the ProbeBw phase for bandwidth utilization balancing.

ProbeBw phase: This operational logic constitutes the core of BBR's congestion control mechanism. The algorithm cyclically modulates transmission rates using a deterministic pacing gain sequence [1.25, 0.75, 1, 1, 1, 1, 1, 1, 1] for bandwidth probing. In the upward probing phase (*pacing_gain* = 1.25), BBR aggressively increases transmission rates to detect unused bandwidth. The subsequent RTT triggers a downward adjustment phase (*pacing_gain* = 0.75), explicitly designed to drain excess queues accumulated during upward probing. For the next six RTT intervals, *pacing_gain* is held at 1 to stabilize transmission rates, with the concurrent congestion window gain (*cwnd_gain*) fixed at 2 throughout these phases to maintain sufficient inflight data for reliable network state inference. A critical

timeout mechanism governs persistent congestion detection: if the minimum RTT sampling process fails to update the RTprop (propagation delay estimate) within 10 s, BBR infers sustained path congestion and transitions to the ProbeRTT phase, initiating aggressive buffer drainage by reducing the congestion window to a single packet.

ProbeRTT phase: In this phase, the congestion window size (CWND) is initialized to 4*MSS with rigorous pacing control, limiting transmissions to four packets per round within a 200 ms timeout threshold to aggressively drain existing network queues while probing for the minimum RTT. Upon timeout expiration, the algorithm evaluates the current network load metrics (e.g., sustained bandwidth/RTT ratios) to determine whether to restart bandwidth probing via the StartUp phase or transition to steady-state bandwidth balancing in the ProbeBw phase, ensuring operational adaptability under dynamic network conditions.

3.2. Motivation

3.2.1. A Discussion on the Phenomenon of Data Over-Injection and the Issue of RTT Unfairness

One of the original design goals of the BBR algorithm is to operate the link's data transmission state at Kleinrock's optimal operating point. This ensures the maximum delivery rate and the minimum delay time (RTprop) while avoiding queue formation and minimizing latency. However, since BDP values differ for flows with varying RTTs, each flow achieves a different optimal operating point on the link. The long RTT flow will occupy a larger portion of the bottleneck bandwidth and buffer due to its higher RTT value. As the RTprop increases, the short RTT flow enters the CWND's bounded phase, allowing the long RTT flow to further seize bandwidth and buffer resources, depriving the short RTT flow of these resources.

Although the BBR congestion control algorithm dynamically adjusts the sending rate by calculating the BDP to proactively avoid excessive data influx, potential overestimation of the bottleneck bandwidth (BtlBw) and underestimation of the RTprop can occur. To compensate for this, BBR sets the CWND value slightly higher than the calculated BDP to buffer ACK delays and queuing within the link. However, this results in an instantaneous rate exceeding the network capacity, especially when multiple streams share the link. Their synchronized probing of bandwidth causes collective data over-injection. Regardless of whether the RTT streams are long or short, the CWND values tend to be slightly larger than the ideal values, leading to a sending rate that surpasses the link's capacity and results in queues shared across all streams.

As shown in Figure 3, when multiple streams with different RTT values compete for the same bottleneck link, the original BBR algorithm operates slightly to the right of the predicted ideal optimal points A and B, reaching the actual operating points C and D.

This results in the total transmission rate exceeding the available bottleneck bandwidth. Over time, this leads to the formation of a persistent queue on the bottleneck link, causing queue-induced delays. Since this persistent queue is shared by all existing streams, long RTT streams, which occupy more bandwidth and buffer space and have higher sending rates, push short RTT streams into a vicious cycle. As fairness continuously deteriorates, long RTT streams eventually become limited by the CWND, but by this point, short RTT streams are left with minimal bandwidth. This situation is highly detrimental to the current scenario, where long RTT streams are prevalent and short RTT streams frequently appear in 5G and Wi-Fi networks.

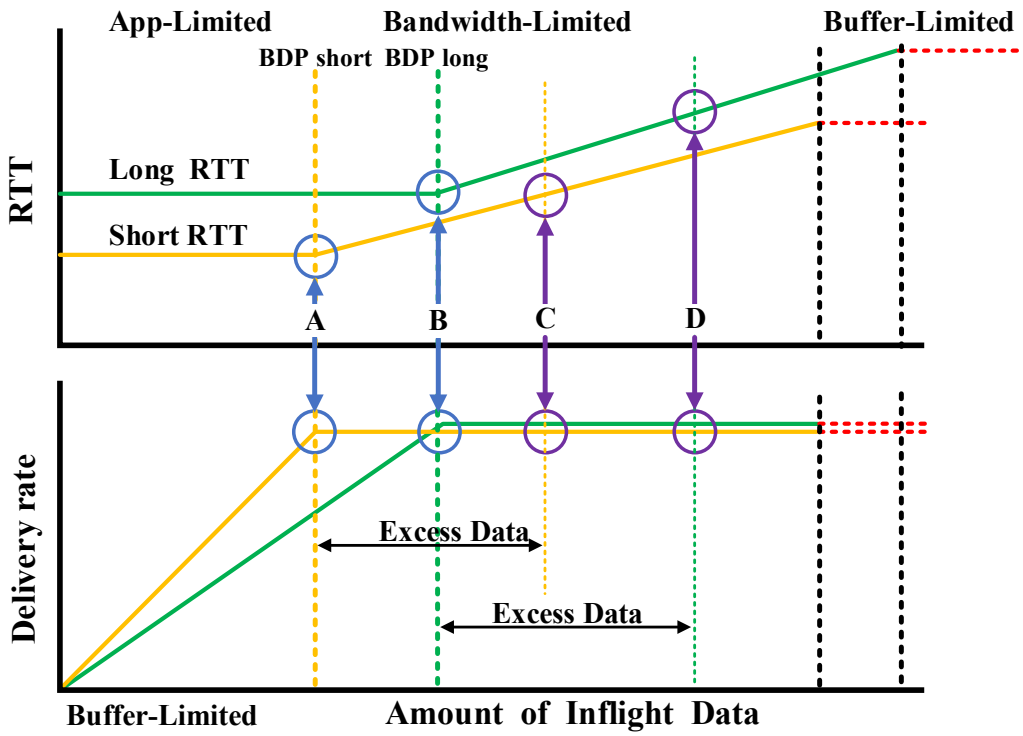


Figure 3. A comparison of the theoretical optimal operating points and the actual operating points for flows with varying RTTs.

Additionally, because BBR is not sensitive to packet loss events, when the bottleneck buffer is small, data over-injection leads to queuing, resulting in a higher retransmission rate on the current link.

3.2.2. Partially Periodic Entry but Meaningless ProbeRTT Phase

To prevent real-time RTT estimation failure caused by prolonged periods without exploration, BBR is designed to automatically enter the ProbeRTT phase if the minimum RTT remains unchanged for ten seconds or if network conditions—such as queuing congestion—necessitate exploration. However, when the link's bottleneck has high bandwidth, traffic is low, or the RTT remains stable, entering the ProbeRTT phase is unnecessary and may lead to a temporary reduction in the sending rate and performance degradation, particularly for long-term connections with steady but low-rate transmissions. Therefore, we propose adapting the ProbeRTT triggering mechanism using time- or event-driven conditions, so that the ProbeRTT phase is initiated only when it is truly needed to detect a new minimum RTT and drain any existing queues on the link.

4. The Optimization Algorithm: Ad-BBR

In this paper, we propose the Ad-BBR algorithm to address these issues by introducing a novel minimum RTT determination mechanism, determining the link state based on the minimum RTT, formulating finer-grained rules for adjusting transmission rates according to different link states, and modifying the trigger logic of the ProbeRTT phase. This approach aims to mitigate over-injection effects, improve the fairness and competition between long and short RTT streams, and enhance the stability of data transmission. Consequently, the fairness between long and short RTT streams is improved, and data transmission stability is enhanced.

4.1. Adaptive Sending Rate Adjustment Mechanism

The adaptive transmit rate regulator employs a novel minimum RTT determination logic to track real-time RTT fluctuations and evaluates the link state based on this new minimum RTT value, thereby achieving more accurate adaptive transmit rate regulation.

4.1.1. Minimum RTT Determination Logic

In high-speed wireless network links, rapid fluctuations in the network state can occur, making it difficult for the RTprop to reflect real-time dynamic changes. Additionally, the BBR congestion control algorithm may fail to update the RTprop correctly during the ProbeBw phase due to continuous packet over-injection. To address this, this paper introduces an adaptive dynamic regulation mechanism with a new RTT (*nRTT*) to replace the old RTprop value for link state determination. By tracking the current RTT in real time and comparing it with the smallest RTT, the *nRTT* is determined. This value is then used to assess the current link state and adjust the transmission state, enabling the algorithm to better handle dynamic links and maintain robust link state determination. The *nRTT* update rule is presented in Algorithm 1.

Algorithm 1. Ad-BBR, minimum RTT determination logic.

```
if measured_rtt ≤ self.α*RTprop then
    nRTT ← RTprop
    over_α_duration = 0.0
else
        over_α_duration ← over_α_duration + dt
        if over_α_duration ≥ 0.3 then
            nRTT ← RTprop
            over_α_duration ← 0.0
    end if
end if
```

4.1.2. Adaptive Transmit Rate Regulation Based on Real-Time Link State

Assume that β is the adaptive adjustment factor applied when queues begin to form, while γ is the factor used under severe queueing conditions. The link bandwidth allocation for RTT streams should be fair under the influence of β and γ. When the link bandwidth is ideally allocated fairly, the bandwidths detected and utilized by two BBR streams with different RTT values should be equal. Given that $BDP = BtlBw^{*}RTprop$, the ratio of BDP for different RTT streams will correspond to the ratio of their RTTs. Let streams 1 and 2 exist, where the RTT obtained from real-time measurements is denoted as *rRTT*, and the possible queuing delay is η_t. These two streams satisfy Equation (8):

$$\frac{BDP_1}{BDP_2} = \frac{rRTT_1}{rRTT_2} = \frac{{\beta_1}^{*}nRTT_1{}^{*}BtlBw_1(t)}{{\beta_2}^{*}nRTT_2{}^{*}BtlBw_2(t)} \tag{8}$$

At this point, where $BtlBw_1 = BtlBw_2$ the following Equation (9) holds:

$$\frac{{\beta_1}^{*}nRTT_1}{{\beta_2}^{*}nRTT_2} = \frac{rRTT_1}{rRTT_2} = \frac{nRTT_1 + \eta_t}{nRTT_2 + \eta_t} \tag{9}$$

This results in Equation (10), which describes the relationship between β and *nRTT*:

$$\beta = g(nRTT) = \frac{nRTT + \eta_t}{nRTT} = 1 + \frac{\eta_t}{nRTT} \tag{10}$$

Since the queuing delay equals the difference between the actual measured RTT and the minimum RTT, i.e., $\eta_t = sRTT - nRTT$, the final expression for β is given by Equation (11):

$$\beta = 1 + \frac{\eta_t}{nRTT} = \frac{nRTT + rRTT - nRTT}{nRTT} = \frac{rRTT}{nRTT} \tag{11}$$

When $rRTT > nRTT$ for a cumulative number of times ≤ 10, when $rRTT \geq nRTT$ for three consecutive times, or when the cumulative count exceeds 10 and the current $rRTT > 1.25nRTT$, this indicates significant queuing on the link with a larger queuing delay. In such cases, a larger γ regulating factor is required to adjust the sending rate. Thus, the regulating factor γ is set as the reciprocal of the queuing delay η_t, as shown in Equation (12):

$$\gamma = \frac{1}{\eta_t} = \frac{1}{rRTT - nRTT} \tag{12}$$

The link reduces the sending rate between different streams by a factor of γ. Due to the varying *rRTT* across streams, the regulation factor γ for the long RTT stream is smaller than that for the short RTT stream. As a result, long RTT streams are more affected by the sending rate adjustment, enabling both long and short RTT streams to empty their queues more effectively. When the short RTT stream quickly empties its queue, link resources become idle, allowing the long RTT stream to gradually resume its sending rate after a period of reduced speed and queue clearing. This adjustment is based on the bottleneck bandwidth size, ensuring the continued transmission of long RTT flow data. The fairness between streams in the link is enhanced by two adaptive adjustment factors, β and γ. The Adaptive Transmit Rate Regulation Based on Real-Time Link State is presented in Algorithm 2.

Algorithm 2. Ad-BBR, adaptive sending rate adjustment mechanism.

```
rRTT ← measured_rtt
if phase ==" ProbeBw" then
  if rRTT < 1.25*nRTT then
    Pacing Rate ← BtlBw*pacing_gain
  else if (1.25*nRTT) ≤ rRTT <(1.5*nRTT) then
    β ← rRTT/nRTT
    Pacing Rate ← BtlBw*Pacing_gain/β
  else
    γ ← 1/(rRTT − nRTT + 1e − 6)
    Pacing Rate ← BtlBw*pacing_gain*γ
end if
end if
```

4.2. Adjustments to the ProbeRTT Phase Trigger Mechanism

The adaptive regulation mechanism in the ProbeBw phase facilitates the formation and elimination of link congestion and introduces a new method for determining the RTprop. This reduces the frequency with which the ProbeRTT phase needs to be entered.

The ProbeRTT phase in BBR proactively reduces the sending rate and forces the network queue to empty in order to measure the current link's minimum RTT (i.e., the RTprop value used to compute the *BDP*). This ensures the algorithm can perform congestion control based on accurate latency information and prevents network changes (e.g., route switching and link quality fluctuations) from resulting in outdated RTprop values. In standard BBR, the ProbeRTT phase is entered every 10 s by default. However, this does not mean that every entry is meaningful during data transmission. In particular, when the network bandwidth is sufficient, there is no competing traffic or queues, the link transmits

low-rate or intermittent traffic, or the latest minimum RTT is naturally observed as RTprop, the entry becomes unnecessary. In such cases, unnecessary entries into the ProbeRTT phase waste bandwidth, reduce the sending rate, and cause instability in data transmission.

Based on the two reasons mentioned above, this paper modifies the ProbeRTT phase-triggering logic. The new mechanism no longer triggers the ProbeRTT phase automatically every ten seconds; instead, it decides whether to enter the phase according to the current link state. By incorporating a dynamic error term derived from the standard deviation of historical RTT data, the algorithm can more accurately determine when to initiate the ProbeRTT phase—taking into account network delay stability and noise levels—without compromising the efficiency of minimum RTT detection. This modification significantly improves transmission stability. To the best of our knowledge, existing studies on improving the ProbeRTT phase trigger logic rarely incorporate dynamic error adjustments based on network delay stability and noise levels to refine the triggering conditions. The revised ProbeRTT phase trigger logic is presented in Algorithm 3.

Algorithm 3. Ad-BBR, the new triggering mechanism for ProbeRTT.

If $length(rt_history) > 1$ **then**
  $rtt_std \leftarrow standard_deviation(rt_history)$
else
  $rtt_std \leftarrow 0.0$
$Y \leftarrow max(X^{*} rtt_std, 5.0)$
if $measured_rtt > RTprop + Y$ **then**
  $phase_timer \leftarrow phase_timer + dt$
  if $phase_timer >= 10.0\ s$ then
    $phase \leftarrow ''ProbeRTT''$
    $phase_timer \leftarrow 0.0$
  end if
else
  $phase_timer \leftarrow 0.0$
end if
else if $phase =''\ ProbeRTT''$ **then**
end if

In this context, Y represents an allowable dynamic error, designed to adapt to varying noise levels in real-world wireless networks. This dynamic value Y is calculated using the standard deviation of historical RTT data, which quantifies the fluctuations in the RTT measurements. This approach enables a better response to network latency stability and noise levels. The formula for Y is shown in Equation (13):

$$Y = max(\ X^{*} RTT_std, Z_min) \tag{13}$$

RTT_std denotes the standard deviation of RTT samples, X is the noise sensitivity factor (set to 0.5 in this paper), and Z_min is the protection threshold (set to 5 ms).

The new trigger logic determines when to enter the ProbeRTT phase by recording the minimum RTT timestamp. Under this mechanism, the ProbeRTT phase is triggered to explore the RTprop when more than 10 s have passed since the condition was first met and the buffer queue size exceeds $Y^{*} BtlBw$ for 10 s, based on the recorded timestamps.

5. Experiment and Evaluation

In this section, we compare the performance of the proposed Ad-BBR algorithm with traditional congestion control algorithms (BBRv1, BBRv3, BBQ, Cubic, and Reno) in terms of throughput, RTT fairness, and transmission stability.

5.1. Simulation Setup

In this paper, the performance of the Ad-BBR algorithm is evaluated using the NS3 network simulation environment, which includes 5G and Wi-Fi simulations. NS3 simulation experiments are conducted by integrating new algorithms and Python APIs into the TCP module of NS3 via TypeId. This integration enables the construction of simulation scenarios—such as multi-stream competition scenarios—using NS3′s Python binding interface, as illustrated in Figure 4. A dumbbell network topology is constructed using NS3′s Python binding interface to simulate multi-stream contention scenarios, with parameters such as RTTs, bottleneck buffer sizes, and bottleneck bandwidths configured according to experimental requirements. The 5G network simulation experiments utilize LteHelper and NrHelper to set up 5G gNB and UE terminals for testing the TCP performance of Ad-BBR under 5G/NR networks. Wi-Fi network simulation experiments employ WifiHelper to configure Wi-Fi AP and STA terminals, evaluating the TCP performance of Ad-BBR under Wi-Fi 802.11n/ac/ax standards. Finally, a Python script iterates over various experimental parameters—including 5G power levels, Wi-Fi rates, and the number of streams—to record experimental results for the throughput, RTT, packet loss rate, fairness index, and other metrics.

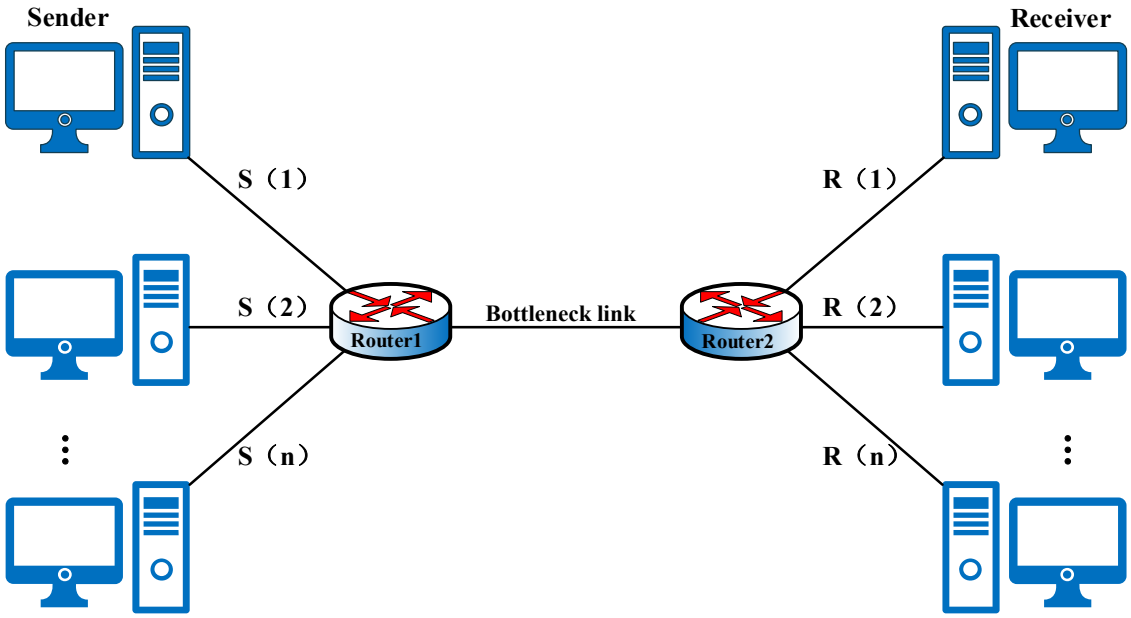


Figure 4. Experimental network topology of the NS3.

5.2. Throughput

To assess whether the Ad-BBR algorithm achieves superior throughput performance, this experiment measures the average throughput and latency of each algorithm using the NS3 simulator. Packet loss rates are set to 0%, 10%, and 20%, and the bottleneck buffers are configured to 0.5 BDP and 5 BDP. Experiments are conducted for each algorithm under six different conditions, and the simulation results are presented in Figure 5.

The experimental results show that as the packet loss rate increases, the throughput of each congestion control algorithm decreases and the delay increases, while increasing the buffer size leads to higher throughput but also greater delay. Packet loss-based congestion control algorithms, such as Cubic and Reno, typically exhibit low throughput and high delay. These algorithms rely on packet loss as a congestion signal; when a loss is detected, they reduce the congestion window and sacrifice throughput. Moreover, they tend to fill the buffer until packet loss occurs, which further increases delay, especially with larger buffers. The BBRv1 algorithm consistently achieves high throughput in most scenarios,

but its latency performance is poor. In contrast, while the BBRv3 and BBQ algorithms perform well under normal conditions, both their throughput and latency deteriorate under high-packet-loss scenarios. In contrast, the Ad-BBR algorithm demonstrates stable and robust throughput across various environments, particularly under high-packet-loss conditions, indicating superior packet loss tolerance. Furthermore, Ad-BBR consistently ranks among the top three in terms of latency performance. Overall, the Ad-BBR algorithm maintains strong performance compared to all other algorithms when packet loss rates or buffer sizes vary. For instance, while its throughput may be slightly lower than that of some algorithms, its latency performance is superior, and it sustains high throughput even when latency is marginally compromised.

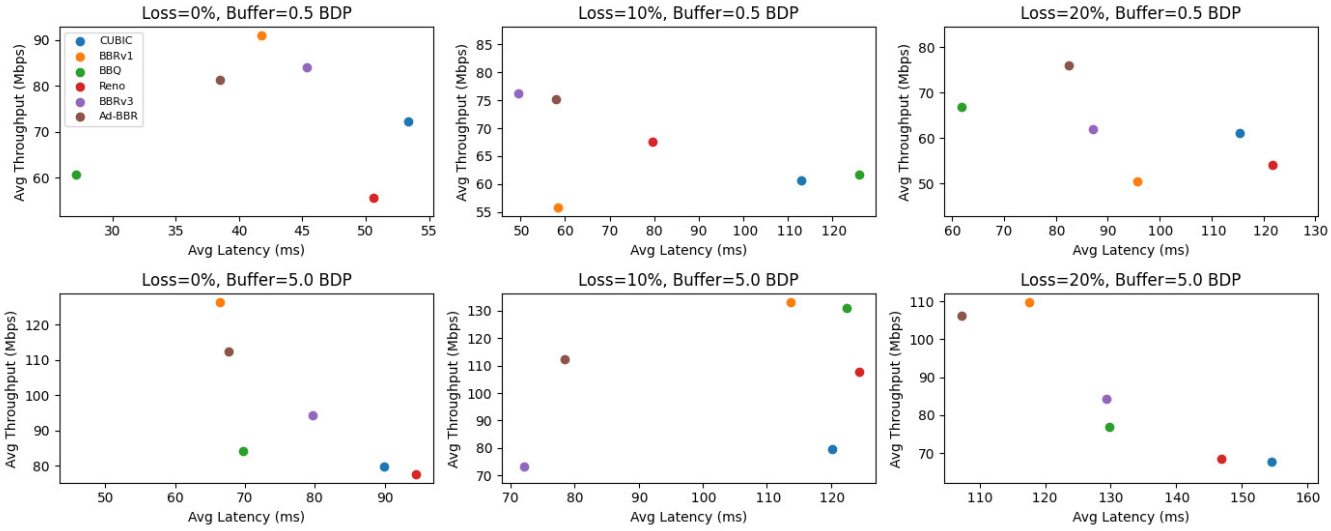


Figure 5. NS3 environment: throughput and latency under different loss and buffer conditions.

Additionally, this experiment configures bottleneck buffers of 0.5 BDPs and 5 BDPs in 5G and Wi-Fi network simulations. A 5 Gbps file is downloaded over a TCP link controlled by different congestion algorithms, and the download rates are recorded. The results are presented in Figures 6 and 7.

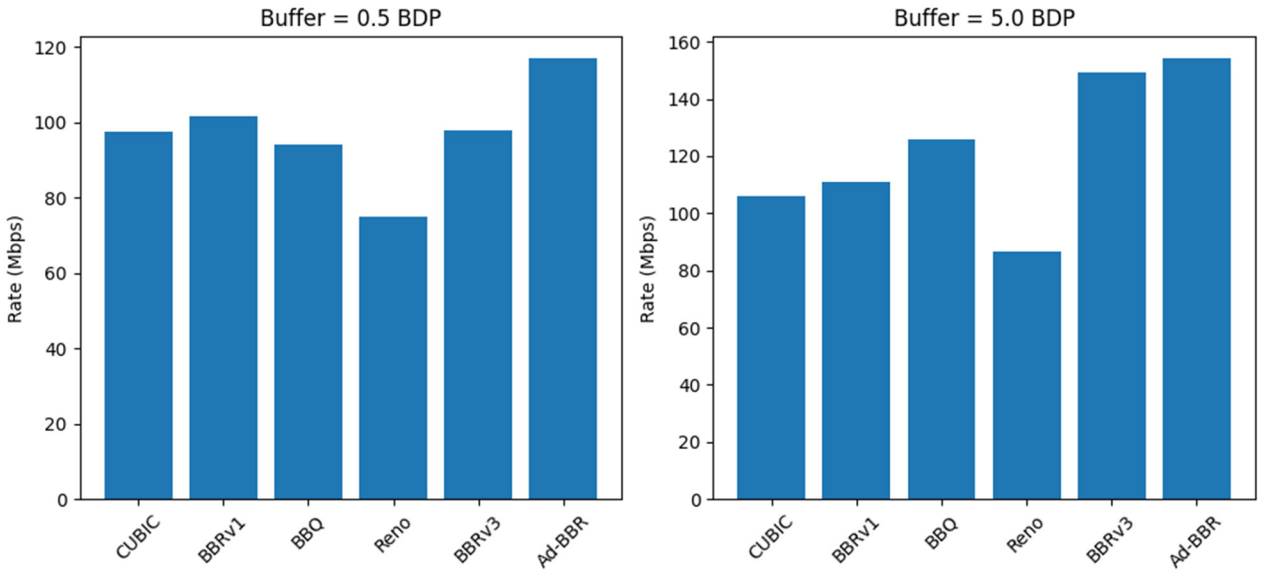


Figure 6. 5G network simulation: download speed comparison in 0.5/5 BDP buffer.

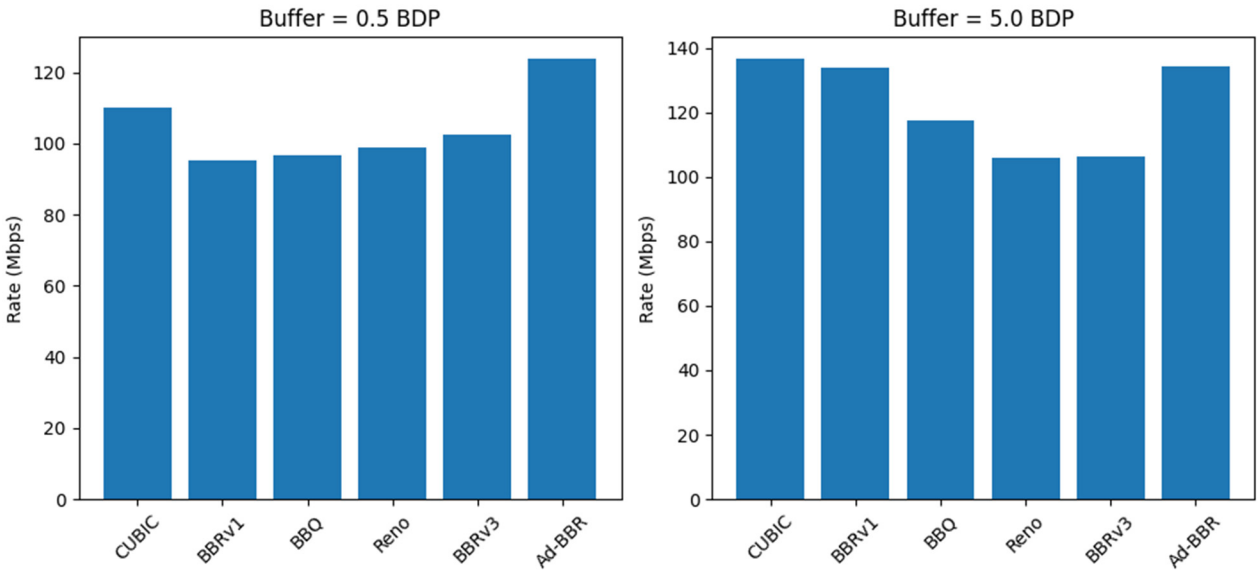


Figure 7. Wi-Fi network simulation: download speed comparison in 0.5/5 BDP buffer.

In both 5G and Wi-Fi environments, increasing the buffer size leads to higher download rates for all algorithms. Notably, BBR-based algorithms achieve significantly greater throughput than packet loss-based algorithms such as Cubic and Reno, particularly in 5G scenarios, where BBR-based algorithms consistently outperform Cubic and Reno. This advantage likely stems from the fact that BBR-based algorithms do not rely solely on packet loss for congestion detection; instead, they perform congestion control by continuously measuring the bandwidth-delay product (BDP) in real time, making them better suited for high-speed wireless networks characterized by abundant bandwidth, random packet loss, and considerable delay fluctuations. Among all evaluated algorithms, Ad-BBR exhibits outstanding performance by achieving download rates that are approximately 17% higher than those of the original BBRv1 and 26% higher than those of BBRv3, an improved version of BBRv1.

5.3. RTT Fairness

To evaluate the fairness improvements of the Ad-BBR algorithm in multi-stream competition scenarios, this section designs two sets of experiments: static network experiments and dynamic network experiments. Throughput performance is a commonly used metric to evaluate RTT fairness in congestion control algorithms under multi-flow competition scenarios. For example, in the case of the BBR algorithm, BBR dynamically adjusts its sending rate by estimating the bottleneck bandwidth and round-trip propagation delay (BDP), which results in flows with different RTTs obtaining different BDP values and, thus, sharing bandwidth unequally, leading to noticeable differences in throughput performance. Moreover, Jain's fairness index, as defined in Equation (14) [33], is used to quantify the fairness performance of each algorithm.

$$J = \left(\frac{(\sum_{i=1}^{n} d_i)^2}{n \sum_{i=1}^{n} {d_i}^2} \right) \tag{14}$$

Here, n represents the number of streams in the link, and denotes the throughput of the i-th stream. The parameter is defined over the interval [0, 1]; a value closer to 1 indicates better bandwidth fairness among the streams.

5.3.1. Static Network Experiments

The static network experiments test the static competitive fairness of Ad-BBR in NS3 simulation and 5G and Wi-Fi simulation network environments, respectively.

In the NS3 simulation experiments, bottleneck buffer sizes of 0.5 BDPs and 5 BDPs were configured. Two streams with round-trip times (RTTs) of 10 ms and 50 ms were introduced simultaneously to compete for a 1 Gbps bottleneck bandwidth. The throughput difference between the two streams was compared under the control of various congestion control algorithms over a duration of 100 s. Jain's fairness index was employed to quantitatively evaluate RTT fairness. Jain's fairness index quantifies the RTT fairness based on the throughput achieved by the 10 ms and 50 ms streams. A value closer to 1 indicates that the throughput of the two streams is more balanced, reflecting a higher level of fairness under the control of the evaluated algorithm.

Figures 8 and 9 display the real-time throughput trends of each algorithm under different buffer sizes, while Figures 10 and 11 present the corresponding average throughput and fairness index results. These figures clearly indicate that as the buffer size increases, the total throughput of all algorithms also increases. For packet loss-based algorithms such as CUBIC and Reno, the 10 ms stream achieves higher throughput than the 50 ms stream; however, the throughput difference between the short and long streams does not significantly increase with larger buffers. Among the BBR-based algorithms, BBRv1 exhibits a dramatic increase in the throughput of the 50 ms stream as the buffer size grows, while the throughput of the 10 ms stream declines, thereby widening the throughput gap and further deteriorating fairness. In contrast, for BBRv3 and BBQ, the fairness difference between the 10 ms and 50 ms streams is smaller; the 10 ms stream throughput under BBRv3 is even slightly higher than that of the 50 ms stream, and the fairness index differences between BBRv3 and BBQ are minimal. Under a 5 BDP buffer, the fairness index of BBQ is slightly higher than that of BBRv3. The Ad-BBR algorithm consistently maintains a narrow average throughput difference. Although its fairness index may not be the highest in every metric, overall, the throughput difference between the 10 ms and 50 ms streams is much smaller, resulting in more stable transmission and a higher overall fairness index in the static fairness tests.

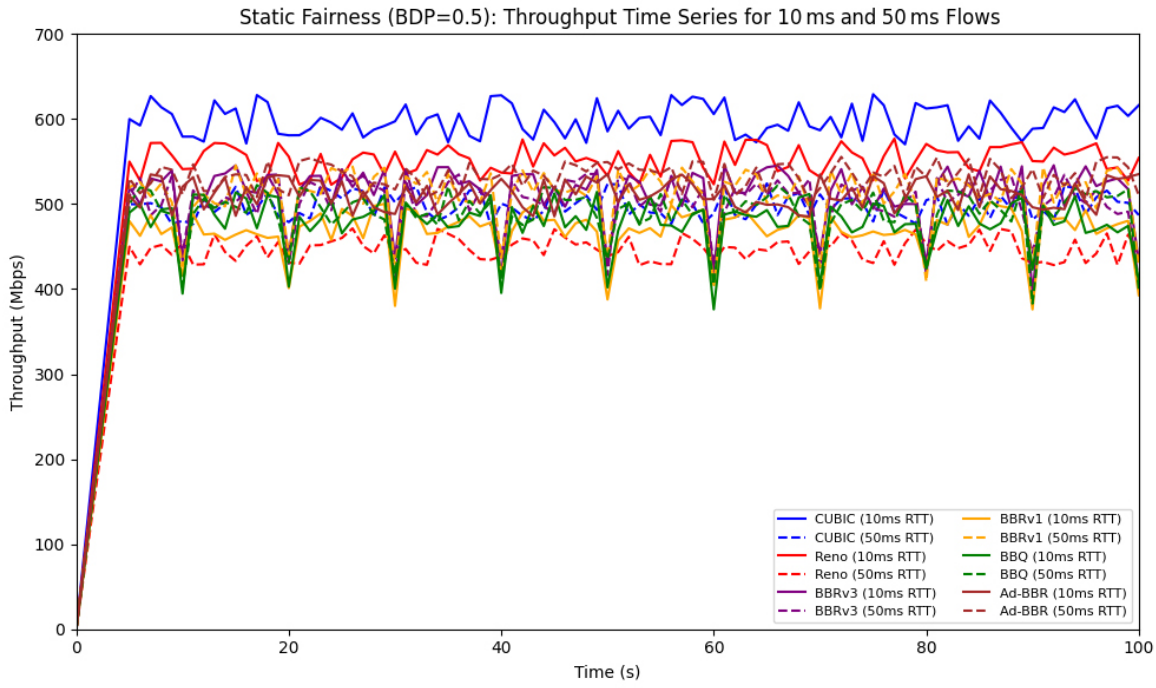


Figure 8. Comparison of real-time throughput for 10 ms RTT flows and 50 ms RTT flows with 0.5 BDP buffer.

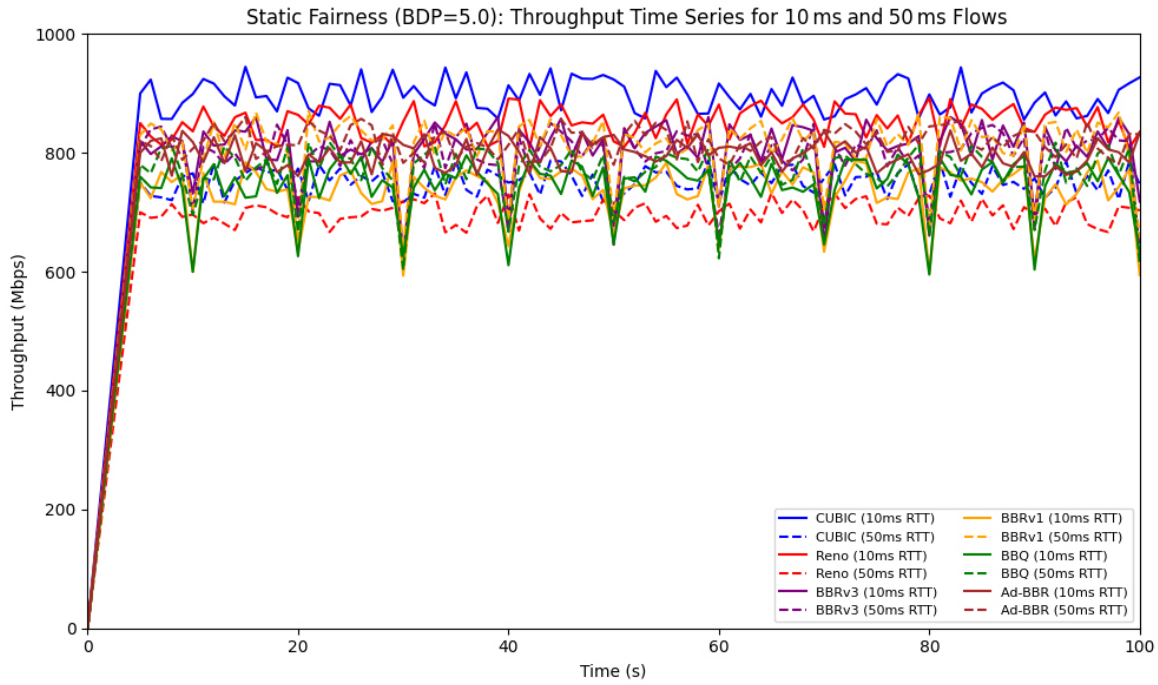


Figure 9. Comparison of real-time throughput for 10 ms RTT flows and 50 ms RTT flows with 5 BDP buffer.

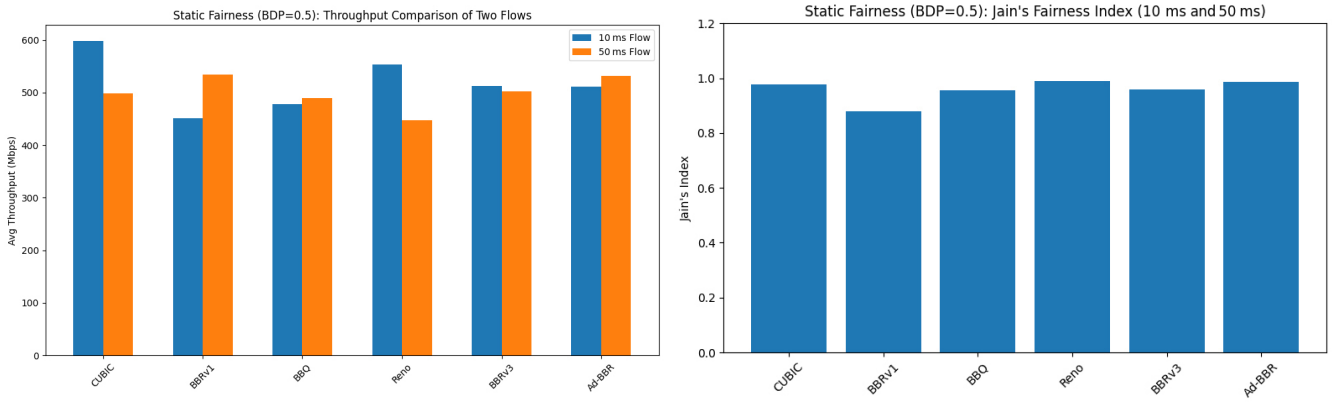


Figure 10. Average throughput and Jain's fairness index for 10 ms RTT flows and 50 ms RTT flows with 0.5 BDP buffer.

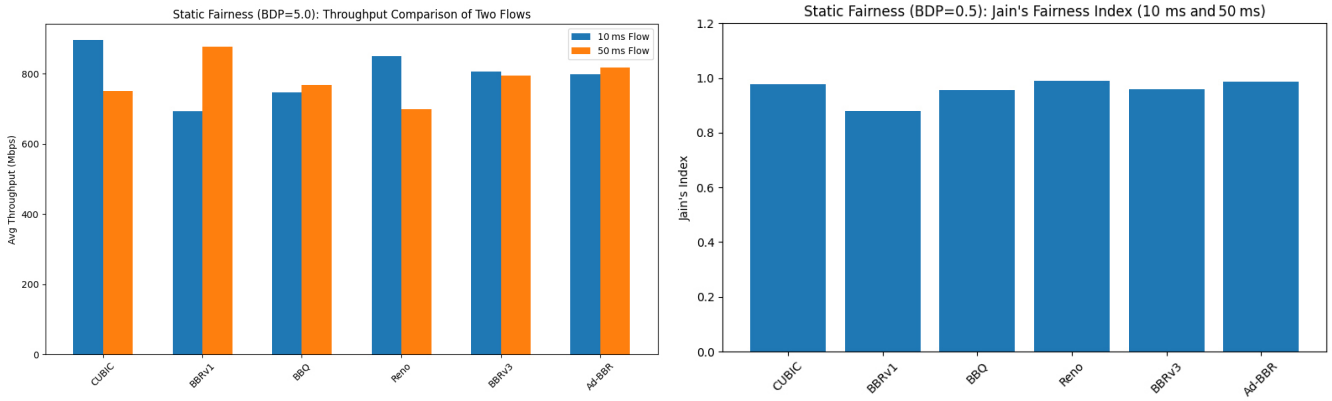


Figure 11. Average throughput and Jain's fairness index for 10 ms RTT flows and 50 ms RTT flows with 5 BDP buffer.

This portion of the static network experiment was repeated in 5G and Wi-Fi simulation environments to evaluate the throughput ratio and fairness index of the 10 ms and 50 ms

streams. The throughput ratio is defined as the proportion of the 10 ms stream's throughput to the total throughput. Since only the 10 ms and 50 ms streams are present on the link, a throughput ratio close to 0.5 indicates that the two streams achieve nearly equal throughput, suggesting that their bandwidth acquisition capability is not significantly influenced by their RTT differences. The experimental results are presented in Figures 12 and 13.

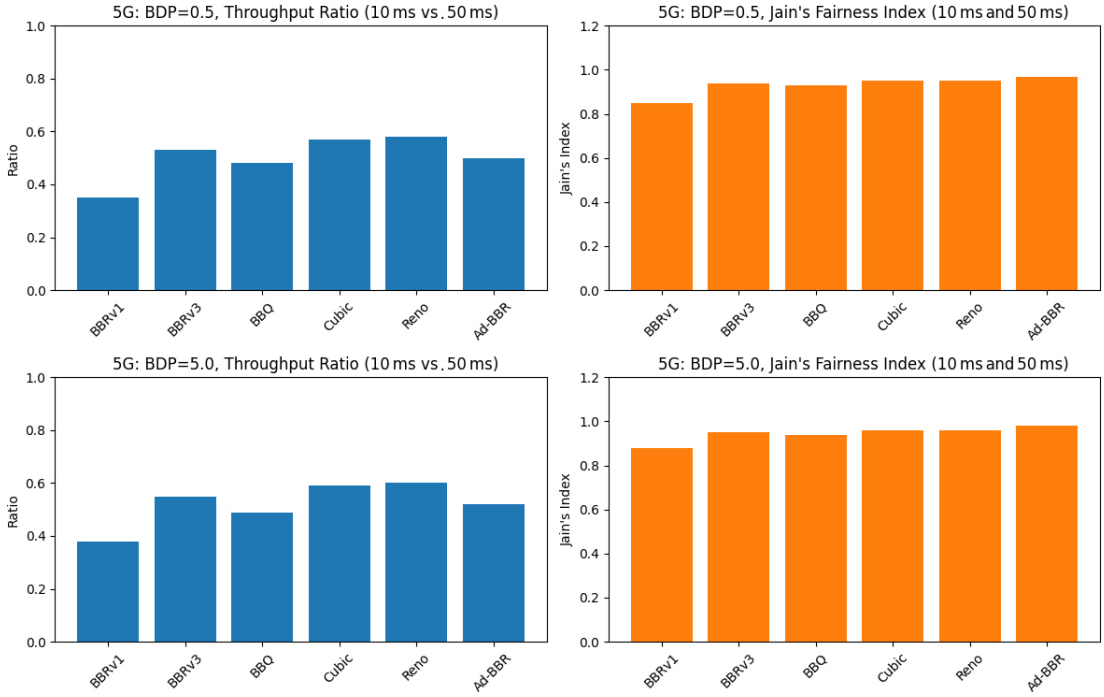


Figure 12. Throughput ratio and Jain's fairness index with 0.5/5 BDP buffer in 5G network simulation.

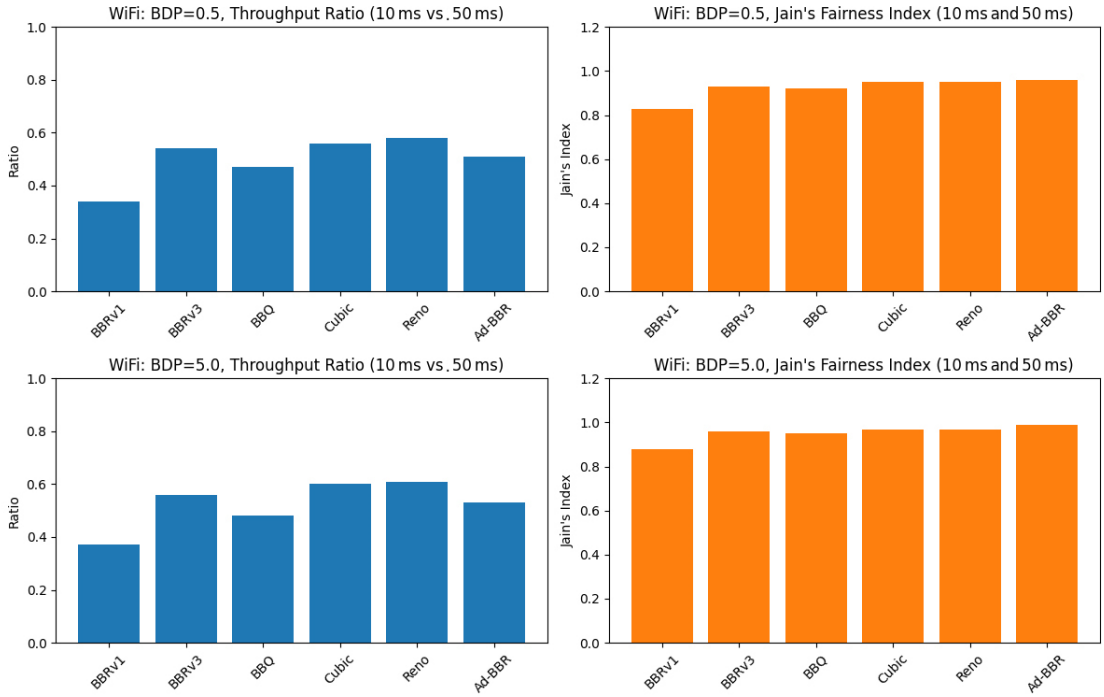


Figure 13. Throughput ratio and Jain's fairness index with 0.5/5 BDP buffer in Wi-Fi network simulation.

In the static network experiments, the BBRv1 algorithm exhibited the most pronounced competitive unfairness in both 5G and Wi-Fi simulation environments, with the throughput of 10 ms streams accounting for only one-third of the total throughput. In contrast, the BBQ

and BBRv3 algorithms demonstrated better fairness, with the 10 ms stream throughput approaching half of the overall throughput, while the Cubic and Reno algorithms favored short streams, achieving a throughput ratio close to 0.6. Although these trends vary slightly with increased buffer sizes, the overall competitive fairness remains largely unchanged. Under the Ad-BBR algorithm, the throughputs of 10 ms and 50 ms streams are relatively balanced, and the fairness indices are significantly improved, resulting in an approximate 11% enhancement in competitive fairness compared to BBRv1 and about 2% better than BBRv3, an improved version that focuses on RTT fairness.

5.3.2. Dynamic Network Experiment

Using NS3, this experiment evaluates the dynamic RTT fairness of the Ad-BBR algorithm. A 20 Mbps bottleneck link is established with an initial 50 ms flow at 0 s. At 30 s, a 10 ms flow is added, followed by a 100 ms flow at 60 s. The 50 ms and 10 ms flows are withdrawn at 90 s and 120 s, respectively. The experiment monitors the change in Jain's fairness index over a 150 s period to assess dynamic RTT fairness, and the results are presented in Figure 14. When only 50 ms streams exist in the link, no competition occurs, so the fairness index for all algorithms remains at 1. At 30 s, when 10 ms streams are introduced, the bandwidth allocation fluctuates and resource contention intensifies, causing the fairness indices of all algorithms to drop steeply. In particular, the fairness indices of BBRv1 and BBQ decline most significantly, while BBRv3 and Ad-BBR perform better. Notably, the fairness index of Ad-BBR quickly recovers to a higher level, indicating that its adaptive sending rate adjustment mechanism effectively mitigates the impact of the new streams. At 60 s, with the addition of a 100 ms stream, the fairness indices of all algorithms begin to fluctuate dramatically. Due to their distinct congestion control principles, the Cubic and Reno algorithms consistently maintain higher fairness indices. Although Ad-BBR initially struggles to adapt to the drastic changes, its fairness index remains relatively stable and quickly recovers after adjustment. Overall, Ad-BBR performs well throughout the dynamic experiment, demonstrating more stable dynamic RTT fairness than other BBR-based algorithms. This makes it a promising solution for maintaining long-term link fairness.

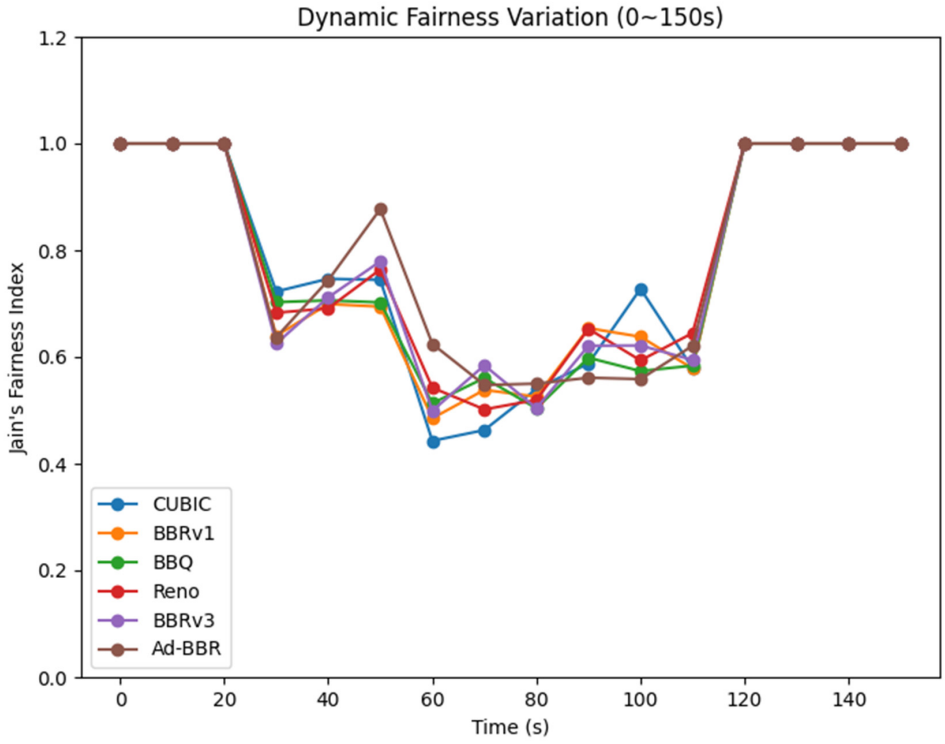


Figure 14. Dynamic fairness variation.

5.4. Retransmission

To evaluate the impact of buffer size and the number of competing streams on the retransmission rate in multi-stream contention scenarios, this experiment configures 1, 2, 4, 8, and 16 streams with bottleneck buffers set to 0.1 BDPs and 1 BDP, respectively, and plots the relationship between retransmission rate and the number of streams for each buffer size.

Figure 15 shows that the retransmission rates of all six algorithms increase as the number of streams rises. Packet loss-based algorithms, such as Cubic and Reno, exhibit lower retransmission rates than BBR-based algorithms. This is likely because with a buffer size of 0.1 BDPs, BBR-based algorithms aggressively multiply the sending rate during proactive bandwidth probing, rapidly filling the buffer and causing quick queue buildup, which leads to higher retransmission rates. When the buffer size increases from 0.1 BDPs to 1 BDP, the retransmission rates decrease slightly, as the larger buffer makes the link more tolerant to bursty traffic. Notably, under multi-stream contention, Ad-BBR consistently maintains a low and stable retransmission rate, achieving approximately a 13% improvement over the original BBRv1 algorithm and about a 7% improvement over BBRv3.

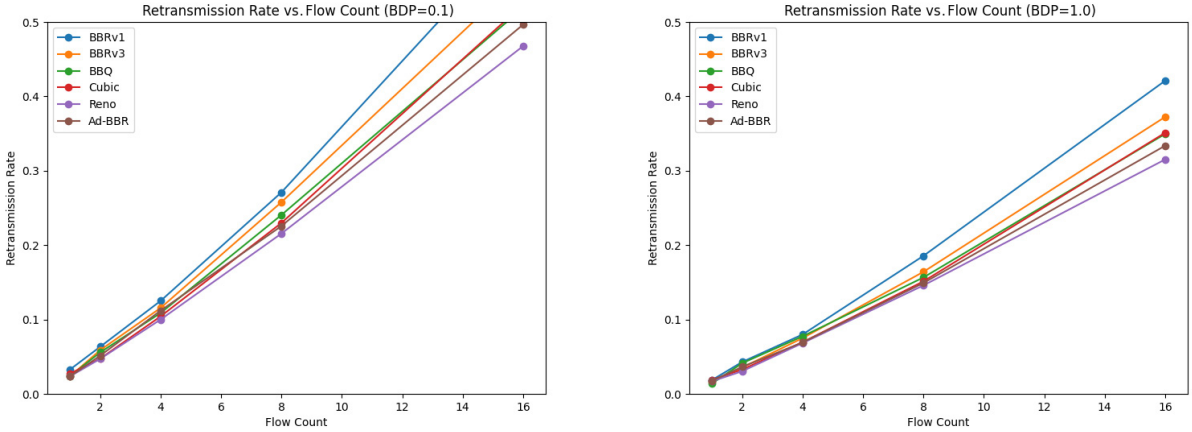


Figure 15. Retransmission rate vs. flow counts with 0.1/1 BDP buffer.

5.5. Latency

This experiment examines the relationship between link delay and the number of competing streams under the Ad-BBR algorithm in a multi-stream contention scenario. The setup involves 1, 2, 4, 8, and 16 streams competing for a 100 Mbps bottleneck bandwidth with a 1 BDP bottleneck buffer. The link delay is observed and plotted as a function of the number of streams, with the results presented in Figure 16.

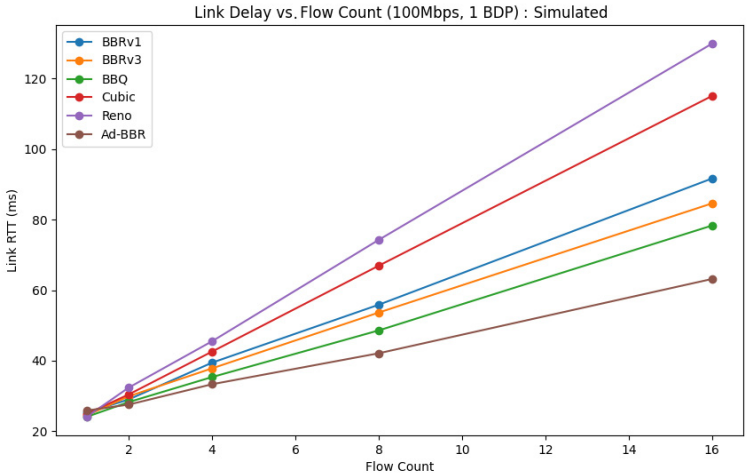


Figure 16. Link delay vs. flow count under (100 Mbps, 1 BDP) conditions.

When there is only one flow in the link, all six algorithms exhibit relatively low latency. As the number of flows increases, the latency rises for all algorithms. However, packet loss-based algorithms like Cubic and Reno experience a sharper and more significant latency increase compared to BBR-related algorithms. This is because Cubic and Reno attempt to achieve higher throughput with limited buffer space, leading to faster queue buildup and increased queuing delays. BBRv1 exhibits more drastic delay variations than improved BBR algorithms due to its data over-injection problem and the dominance of long RTT flows. Among them, the BBQ algorithm achieves better queue delay reduction through its joint bandwidth- and queue-awareness mechanism, making its queue delay more stable and lower than BBRv3 in this experiment. Compared to other algorithms, Ad-BBR demonstrates more stable and minimal delay variations. Its delay increase with the growing number of streams is the smallest, and it maintains the lowest latency for a given number of streams. This performance represents a ~20% improvement over BBRv1 and represents a ~17% improvement over BBRv3, highlighting the importance of reducing data over-injection and enhancing RTT fairness in minimizing queue delays.

6. Conclusions

In this paper, we propose the Ad-BBR algorithm to address the issues of data over-injection, unfairness between long and short RTT streams, and transmission instability caused by the periodic ProbeRTT phase observed in the BBR algorithm within multi-stream concurrent scenarios in high-speed wireless networks.

The Ad-BBR algorithm determines the value of *nRTT* by tracking the size relationship between the current RTT and the previous minimum RTT in real time to determine the current link state, and based on the *nRTT* and the real-time link state, it distinguishes the transmission rate adjustment rules corresponding to different link states with finer granularity so as to reduce the phenomenon of over-injection of data and improve the fairness of the RTT in the multi-stream competition scenario. In addition, the new ProbeRTT phase-triggering mechanism determines whether to enter the phase based on the link state, which reduces the frequency of meaningless ProbeRTT phases and greatly improves the transmission stability.

The experimental results indicate that compared with BBRv1, Ad-BBR demonstrates more stable and smoother data transmission in multi-stream competition scenarios. Specifically, its throughput increases by 17%, RTT fairness improves by approximately 30%, the retransmission rate decreases by 13%, and latency is reduced by about 20%. Overall, Ad-BBR outperforms several comparative algorithms across all key performance metrics.

Future work may explore extracting state-space parameters from real network environments to train reinforcement learning models. By leveraging reinforcement learning, the agent can dynamically determine key parameters that affect BBR performance—such as the congestion window, sending rate, exploration interval, and ProbeRTT phase trigger conditions—thus enabling more adaptive and efficient congestion control. Furthermore, after completing simulation experiments, real network tests will be conducted to evaluate the reasonableness of the algorithm's complexity and to verify the consistency of the performance improvements.

Author Contributions: Conceptualization, M.W., X.Z. and M.G.; methodology, M.W., X.Z. and M.G.; software, X.Z. and M.G.; validation, X.Z. and M.G.; writing—original draft preparation, X.Z.; writing—review and editing, M.W., X.Z. and M.G.; visualization, M.W., X.Z. and M.G.; supervision, M.W., X.Z. and M.G.; project administration, M.W., X.Z., F.J. and M.G. All authors have read and agreed to the published version of the manuscript.

Funding: This study was funded by the Information and Communication Business Cost Project of JSCMC, Grant/Award Number SXJD-XXTXF-202303, and the Scientific Research Plan of JSLLKYJH, Grant/Award Number 23-JSLLKY011.

Data Availability Statement: The raw data supporting the conclusions of this article will be made available by the authors on request.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Yang, Y.R.; Lam, S.S. General AIMD congestion control. In Proceedings of the 2000 International Conference on Network Protocols, Osaka, Japan, 14–17 November 2000; IEEE: Piscataway, NJ, USA, 2000; pp. 187–198.
2. Kesselman, A.; Mansour, Y. Adaptive AIMD congestion control. In Proceedings of the of the Twenty-Second Annual Symposium on Principles of Distributed Computing, Boston, MA, USA, 13–16 July 2003; pp. 352–359.
3. Cardwell, N.; Cheng, Y.; Gunn, C.S.; Yeganeh, S.H.; Jacobson, V. Bbr: Congestion-based congestion control: Measuring bottleneck bandwidth and round-trip propagation time. *Queue* **2016**, *14*, 20–53. [CrossRef]
4. Lim, H.; Lee, J.; Lee, J.; Sathyanarayana, S.D.; Kim, J.; Nguyen, A.; Kim, K.T.; Im, Y.; Chiang, M.; Grunwald, D.; et al. An empirical study of 5G: Effect of edge on transport protocol and application performance. *IEEE Trans. Mob. Comput.* **2023**, *23*, 3172–3186. [CrossRef]
5. Chaccour, C.; Soorki, M.N.; Saad, W.; Bennis, M.; Popovski, P.; Debbah, M. Seven defining features of terahertz (THz) wireless systems: A fellowship of communication and sensing. *IEEE Commun. Surv. Tutor.* **2022**, *24*, 967–993. [CrossRef]
6. Zheng, S.; Liu, J.; Yan, X.; Xing, Z.; Di, X.; Qi, H. BBR-R: Improving BBR performance in multi-flow competition scenarios. *Comput. Netw.* **2024**, *254*, 110816. [CrossRef]
7. Hock, M.; Bless, R.; Zitterbart, M. Experimental evaluation of BBR congestion control. In Proceedings of the 2017 IEEE 25th International Conference on Network Protocols (ICNP), Toronto, ON, Canada, 10–13 October 2017; IEEE: Piscataway, NJ, USA, 2017; pp. 1–10.
8. Ware, R.; Mukerjee, M.K.; Seshan, S.; Sherry, J. Modeling BBR's interactions with loss-based congestion control. In Proceedings of the Internet Measurement Conference, Amsterdam The Netherlands, 21–23 October 2019; pp. 137–143.
9. Jain, R.K.; Chiu, D.M.W.; Hawe, W.R. Congestion Control for Low-Latency Interactive Video Streaming. U.S. Patent Application 17/712,476, 5 October 2023.
10. Drucker, R.; Baraskar, G.; Balasubramanian, A.; Gandhi, A. Bbr vs. bbrv2: A performance evaluation. In Proceedings of the 2024 16th International Conference on Communication Systems & NETworkS (COMSNETS), Bengaluru, India, 3–7 January 2024; IEEE: Piscataway, NJ, USA, 2024; pp. 379–387.
11. Zeynali, D.; Weyulu, E.N.; Fathalli, S.; Chandrasekaran, B.; Feldmann, A. BBRv3: Algorithm bug fixes and public internet deployment. In Proceedings of the IETF Meeting, Vancouver, BC, Canada, 23 July 2023; pp. 1–36.
12. Ma, S.; Jiang, J.; Wang, W.; Li, B. Fairness of congestion-based congestion control: Experimental evaluation and analysis. *arXiv* **2017**, arXiv:1706.09115.
13. Yang, M.; Yang, P.; Wen, C.; Liu, Q.; Luo, J.; Yu, L. Adaptive-BBR: Fine-grained congestion control with improved fairness and low latency. In Proceedings of the 2019 IEEE Wireless Communications and Networking Conference (WCNC), Marrakech, Morocco, 15–18 April 2019; IEEE: Piscataway, NJ, USA, 2019; pp. 1–6.
14. Scherrer, S.; Legner, M.; Perrig, A.; Schmid, S. Model-based insights on the performance, fairness, and stability of BBR. In Proceedings of the 22nd ACM Internet Measurement Conference, Nice, France, 25–27 October 2022; pp. 519–537.
15. Xu, J.; Pan, W.; Tan, H.; Cheng, L.; Li, X.; Li, X. A Modified TCP BBR to Enable High Fairness in High-Speed Wireless Networks. *Future Internet* **2024**, *16*, 392. [CrossRef]
16. Jaeger, B.; Scholz, D.; Raumer, D.; Geyer, F.; Carle, G. Reproducible measurements of TCP BBR congestion control. *Comput. Commun.* **2019**, *144*, 31–43. [CrossRef]
17. Kim, G.H.; Mahmud, I.; Cho, Y.Z. Fairness improvement of BBR congestion control algorithm for different RTT flows. In Proceedings of the 2019 International Conference on Electronics, Information, and Communication (ICEIC), Auckland, New Zealand, 22–25 January 2025; IEEE: Piscataway, NJ, USA, 2019; pp. 1–2.
18. Mahmud, I.; Kim, G.H.; Lubna, T.; Cho, Y.Z. BBR-ACD: BBR with advanced congestion detection. *Electronics* **2020**, *9*, 136. [CrossRef]
19. Kim, G.H.; Cho, Y.Z. Delay-aware BBR congestion control algorithm for RTT fairness improvement. *IEEE Access* **2019**, *8*, 4099–4109. [CrossRef]
20. Pan, W.; Tan, H.; Li, X.; Li, X. Improved RTT fairness of BBR congestion control algorithm based on adaptive congestion window. *Electronics* **2021**, *10*, 615. [CrossRef]

21. Njogu, C.K.; Yang, W.; Njogu, H.W.; Bosire, A. BBR-With Enhanced Fairness (BBR-EFRA): A new enhanced RTT fairness for BBR congestion control algorithm. *Comput. Commun.* **2023**, *200*, 95–103. [CrossRef]
22. Xie, Y.; Jiang, X.; Gong, G.; Jiang, Z.; Jin, G.; Chen, H. Yinker: A flexible BBR to achieve the high-throughput and low-latency data transmission over Wi-Fi and 5G networks. *Comput. Netw.* **2023**, *222*, 109530. [CrossRef]
23. Deng, Z.; Liu, Y.; Liu, J.; Argyriou, A.; Liu, D. BBR-based and fairness-guaranteed congestion control and packet scheduling for MPQUIC over heterogeneous networks. *Comput. Commun.* **2024**, *224*, 213–224. [CrossRef]
24. Tong, V.; Souihi, S.; Tran, H.A.; Mellouk, A. Troubleshooting solution for traffic congestion control. *J. Netw. Comput. Appl.* **2024**, *229*, 103923. [CrossRef]
25. Han, Z.; Hasegawa, G. Overcoming Fairness and Latency Challenges in BBR with an Adaptive Delay Detection. *IEEE Access* **2025**, *13*, 37318–37327. [CrossRef]
26. Kihungi Njogu, C.; Yang, W.; Waita Njogu, H.; Bosire, A. BBR-with enhanced bandwidth estimation (BBR-EBE+): An improved BBR congestion control algorithm based on TCP acknowledgment compression and aggregation. *Telecommun. Syst.* **2025**, *88*, 1–13. [CrossRef]
27. Neal Cardwell, Y.C.; Yang, K.; Pj, S.H.Y.; Seung, Y.; Hsiao, L.; Mathis, M.; Jacobson, V. BBRv2: A model-based congestion control. In Proceedings of the IETF Meeting, Online, 8 November 2021.
28. Kfoury, E.F.; Gomez, J.; Crichigno, J.; Bou-Harb, E. An emulation-based evaluation of TCP BBRv2 alpha for wired broadband. *Comput. Commun.* **2020**, *161*, 212–224. [CrossRef]
29. Nandagiri, A.; Tahiliani, M.P.; Misra, V.; Ramakrishnan, K.K. BBRvl vs BBRv2: Examining performance differences through experimental evaluation. In Proceedings of the 2020 IEEE International Symposium on Local and Metropolitan Area Networks LANMAN, Orlando, FL, USA, 13–15 July 2020; IEEE: Piscataway, NJ, USA, 2020; pp. 1–6.
30. Zeynali, D.; Weyulu, E.N.; Fathalli, S.; Chandrasekaran, B.; Feldmann, A. Promises and Potential of BBRv3. In *International Conference on Passive and Active Network Measurement*; Springer Nature: Cham, Switzerland, 2024; pp. 249–272.
31. Mascolo, S.; Casetti, C.; Gerla, M.; Sanadidi, M.Y.; Wang, R. TCP Westwood: Bandwidth estimation for enhanced transport over wireless links. In Proceedings of the 7th Annual International Conference on Mobile Computing and Networking, Rome, Italy, 16–21 July 2001; pp. 287–297.
32. Kleinrock, L. Power and deterministic rules of thumb for probabilistic problems in computer communications. In Proceedings of the ICC 1979 International Conference on Communications, Boston, MA, USA, 10–14 June 1979; Volume 3, pp. 43.1.1–43.1.10.
33. Jain, R.K.; Chiu, D.M.W.; Hawe, W.R. *A Quantitative Measure of Fairness and Discrimination*; Eastern Research Laboratory, Digital Equipment Corporation: Hudson, MA, USA, 1984; Volume 21.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.